

JavaScript Practical 06

Hoisting & Closures

Name: Nikhil kumar

Course: B.C.A

Semester: 5th Semester

Part 1 — Hoisting with var

Question: Task 1.1: Predict and run the output.

Answer: The variable is hoisted, so its value is undefined before assignment.

```
console.log(city);  
var city = "Haridwar";  
console.log(city);
```

Output:

```
undefined  
Haridwar
```

Question: Task 1.2: What does this print?

Answer: The local var message is hoisted to the top of the function.

```
function showMessage() {  
    console.log(message);  
    var message = "Hello";  
    console.log(message);  
}  
showMessage();
```

Output:

```
undefined  
Hello
```

Question: Task 1.3: Why is the first line not "global"?

Answer: The local var name is hoisted inside test(), so it hides the global variable.

```
var name = "global";  
function test() {  
    console.log(name);  
    var name = "local";  
}
```

```
test();
```

Output:

```
undefined
```

Question: Task 1.4: Write a program that prints undefined and then your favourite food.

Answer: I used var because its declaration is hoisted.

```
console.log(food);  
var food = "Pizza";  
console.log(food);
```

Output:

```
undefined  
Pizza
```

Part 2 — Function Hoisting

Question: Task 2.1: What error appears?

Answer: A function expression stored in var is undefined before assignment, so calling it gives a TypeError.

```
sayHi();  
var sayHi = function () {  
    console.log("Hi!");  
};
```

Output:

```
TypeError: sayHi is not a function
```

Question: Task 2.2: Change var to const. Does the error change?

Answer: Yes. With const, the variable is in the TDZ, so the error becomes a ReferenceError.

```
var → TypeError: sayHi is not a function  
const → ReferenceError: Cannot access 'sayHi' before initialization
```

Question: Task 2.3: Complete the sorting table.

Answer: Function declarations are hoisted completely. Function expressions follow the variable rules.

function a() {}	→ Yes	→ No error
var b = function() {}	→ No	→ TypeError
const c = () => {}	→ No	→ ReferenceError
let d = function() {}	→ No	→ ReferenceError

Question: Task 2.4: Two functions with the same name.

Answer: The later function declaration wins.

```
console.log(fnA());  
function fnA() { return "First"; }  
function fnA() { return "Second"; }
```

Output:

Second

Question: Task 2.5: Write a top-down story using helper functions at the bottom.

Answer: It works because function declarations are hoisted completely.

```
wakeUp();  
eatBreakfast();  
goToCollege();  
  
function wakeUp() { console.log("Wake up"); }  
function eatBreakfast() { console.log("Eat breakfast"); }  
function goToCollege() { console.log("Go to college"); }
```

Part 3 — let, const and the Temporal Dead Zone

Question: Task 3.1: Use const PI after console.log(PI).

Answer: const is hoisted but cannot be used before initialization.

```
console.log(PI);  
const PI = 3.14;
```

Output:

ReferenceError: Cannot access 'PI' before initialization

Question: Task 3.2: Compare typeof with var and let.

Answer: var gives undefined, while let gives a ReferenceError because of the TDZ.

```
console.log(typeof x);  
var x = 5;  
console.log(typeof y);  
let y = 5;
```

Output:

undefined

ReferenceError: Cannot access 'y' before initialization

Question: Task 3.3: Complete the sentence.

Answer: undefined occurs when var is hoisted without its value; ReferenceError occurs when let/const is used in the TDZ; TypeError occurs when an undefined value is called like a function.

Question: Task 3.4: Write four mini-programs.

Answer: These examples show the four different results.

```
console.log(a);
```

```
var a = 5;
```

```
console.log(b);
```

```
let b = 5;
```

```
hello();
```

```
var hello = function() {};
```

```
console.log(add(2, 3));
```

```
function add(a, b) { return a + b; }
```

Output:

```
undefined
```

```
ReferenceError
```

```
TypeError
```

```
5
```

Part 4 — Your First Closure

Question: Task 4.1: Create counters and call them.

Answer: Each call to makeCounter() creates a separate count, so counterB starts from 1.

```
function makeCounter() {
```

```
  let count = 0;
```

```
  return function() {
```

```
    count++;
```

```
    return count;
```

```
  };
```

```
}
```

```
const counterA = makeCounter();
```

```
const counterB = makeCounter();
```

```
console.log(counterA(), counterA(), counterA(), counterA(), counterA());
```

```
console.log(counterB(), counterB());
```

Output:

```
1 2 3 4 5  
1 2
```

Question: Task 4.2: Try console.log(count) outside the function.

Answer: It gives a ReferenceError. This proves that count is private to the closure.

```
ReferenceError: count is not defined
```

Question: Task 4.3: Complete the multiplier factory.

Answer: The inner function remembers n from the outer function.

```
function makeMultiplier(n) {  
  return function(x) {  
    return x * n;  
  };  
}  
  
const double = makeMultiplier(2);  
const triple = makeMultiplier(3);  
console.log(double(5), triple(5));
```

Output:

```
10 15
```

Question: Task 4.4: Write makeGreeter(greeting).

Answer: The inner function remembers the greeting value.

```
function makeGreeter(greeting) {  
  return function(name) {  
    console.log(greeting + ", " + name + "!");  
  };  
}  
  
makeGreeter("Namaste")("Aditi");
```

Output:

```
Namaste, Aditi!
```

Question: Task 4.5: Make a chai counter.

Answer: Each counter keeps its own cup count.

```
function makeCupCounter() {  
  let cups = 0;
```

```

    return function() {
        cups++;
        return "Cup number " + cups + " of chai";
    };
}

const friendA = makeCupCounter();
const friendB = makeCupCounter();
console.log(friendA());
console.log(friendA());
console.log(friendB());

```

Output:

```

Cup number 1 of chai
Cup number 2 of chai
Cup number 1 of chai

```

Part 5 — Private Data with Closures

Question: Task 5.1: Try to change wallet.balance.

Answer: The private balance does not change because it is stored inside the closure.

```

wallet.balance = 99999;
console.log(wallet.show());

```

Output:

```

The original balance remains unchanged.

```

Question: Task 5.2: Add a reset() function.

Answer: The start value is remembered by the closure, so reset can restore the original balance.

```

function createWallet(start) {
    let balance = start;
    return {
        add(n) { balance += n; return balance; },
        spend(n) {
            if (n > balance) return "Insufficient balance";
            balance -= n; return balance;
        },
        show() { return balance; },
        reset() { balance = start; return balance; }
    };
}

```

```
};  
}
```

Question: Task 5.3: Complete limiter(max).

Answer: The used variable is private and remembers how many attempts were made.

```
function limiter(max) {  
  let used = 0;  
  return function() {  
    if (used < max) {  
      used++;  
      return "Attempt " + used + " of " + max;  
    }  
    return "Locked!";  
  };  
}  
  
const tryLogin = limiter(3);  
console.log(tryLogin());  
console.log(tryLogin());  
console.log(tryLogin());  
console.log(tryLogin());
```

Output:

```
Attempt 1 of 3  
Attempt 2 of 3  
Attempt 3 of 3  
Locked!
```

Question: Task 5.4: Create a private diary.

Answer: The entries array is private, so it cannot be accessed directly.

```
function createDiary() {  
  const entries = [];  
  return {  
    write(text) { entries.push(text); },  
    read() { return [...entries]; }  
  };  
}  
  
const diary = createDiary();  
diary.write("Study JavaScript");
```

```
console.log(diary.read());  
console.log(diary.entries);
```

Output:

```
["Study JavaScript"]  
undefined
```

Part 6 — Closures in Loops

Question: Task 6.1: Explain the outputs.

Answer: var uses one shared i, which ends at 3, so all functions return 3. let creates a separate value for each loop iteration.

```
const withVar = [];  
for (var i = 0; i < 3; i++) withVar.push(() => i);  
console.log(withVar.map(f => f()));
```

```
const withLet = [];  
for (let j = 0; j < 3; j++) withLet.push(() => j);  
console.log(withLet.map(f => f()));
```

Output:

```
[3, 3, 3]  
[0, 1, 2]
```

Question: Task 6.2: Predict the timer output.

Answer: The var loop prints 4 three times because the same k is shared. The let loop keeps separate values.

```
for (var k = 1; k <= 3; k++) {  
  setTimeout(() => console.log("var:", k), 1000);  
}  
for (let m = 1; m <= 3; m++) {  
  setTimeout(() => console.log("let:", m), 1000);  
}
```

Output:

```
var: 4  
var: 4  
var: 4  
let: 1  
let: 2
```

let: 3

Question: Task 6.3: Fix the bug by changing one word.

Answer: Change var to let.

Part 7 — Mini Project: Smart Wallet with Login Guard

Question: Question: Create the smart wallet using closures and hoisting.

Answer: I used function declarations so the main code can be placed at the top. The wallet keeps balance and history private, and the login guard limits spend attempts.

```
console.log("Add:", wallet.add(200));
console.log("Spend:", spendWithGuard(150));
console.log("Spend:", spendWithGuard(1000));
console.log("Balance:", wallet.show());
console.log("History:", wallet.history());

function createWallet(start) {
  let balance = start;
  const records = [];
  return {
    add(n) { balance += n; records.push("Added " + n); return balance; },
    spend(n) {
      if (n > balance) return "Insufficient balance";
      balance -= n; records.push("Spent " + n); return balance;
    },
    show() { return balance; },
    history() { return records; }
  };
}

function limiter(max) {
  let used = 0;
  return function() {
    if (used < max) { used++; return true; }
    return false;
  };
}

const wallet = createWallet(500);
const guard = limiter(3);
```

```
function spendWithGuard(amount) {  
  if (!guard()) return "Locked!";  
  return wallet.spend(amount);  
}
```

Output:

```
Add: 700  
Spend: 550  
Spend: Insufficient balance  
Balance: 550  
History: ["Added 200", "Spent 150"]
```

Question: Bonus: Create makeDiscount(percent).

Answer: The returned function remembers the discount percentage.

```
function makeDiscount(percent) {  
  return function(price) {  
    return price - (price * percent / 100);  
  };  
}  
  
const festive = makeDiscount(10);  
console.log(festive(500));
```

Output:

```
450
```

Part 8 — Debugging Challenge

Question: Snippet 1: Make the undefined go away.

Answer: The value should be assigned before it is printed.

```
var total = 5;  
console.log(total);
```

Output:

```
5
```

Question: Snippet 2: Fix the TypeError without changing var.

Answer: The function expression must be assigned before it is called.

```
var greet = function() {  
  console.log("Hi");  
};
```

```
greet();
```

Output:

```
Hi
```

Question: Snippet 3: Fix the counter.

Answer: The function should return another function so that the value is remembered.

```
function makeCounter() {  
  let c = 0;  
  return function() {  
    return ++c;  
  };  
}  
  
const next = makeCounter();  
console.log(next(), next());
```

Output:

```
1 2
```

Question: Snippet 4: Fix the counter.

Answer: count must be outside the returned function so it is not recreated on every call.

```
function makeCounter2() {  
  let count = 0;  
  return function() {  
    count++;  
    return count;  
  };  
}  
  
const n = makeCounter2();  
console.log(n(), n(), n());
```

Output:

```
1 2 3
```

Conclusion

In this practical, I learned hoisting, function hoisting, the TDZ, closures, private data, and closures in loops. I also practiced fixing common JavaScript errors.